Ira A. Fulton Schools of Engineering
Arizona State University

CSE 540: Engineering Blockchain Application

Blockchain-Based Supply Chain Provenance System Project

Purpose

The Blockchain-Based Supply Chain Provenance System project focuses on designing a decentralized solution to improve transparency, traceability, and trust across supply chain networks. By applying distributed ledger technology and smart contracts, this project explores how blockchain can address common industry challenges such as fragmented data, limited visibility, and risks of fraud or tampering. Within the context of the course, this topic reinforces core concepts in system design, smart contract development, and stakeholder interaction while highlighting a widely recognized industry use case. It provides practical insight into how blockchain can support accountability and data integrity in complex, multi-party environments.

Objectives

Learners will be able to:

- **1.1**: Describe the basic blockchain data structure
- **1.4**: Describe applications of blockchain technology
- **1.5**: Analyze the viability of a current blockchain application
- **4.2**: Interpret basic and advanced scripts
- **4.3**: Create basic and advanced scripts
- **6.2**: Explain important consensus properties for blockchain networks
- **6.3**: Describe several consensus algorithms at a fundamental level
- **8.1**: Identify the different node types in a cryptocurrency/blockchain network
- **8.2**: Describe how nodes discover the network (other peers)
- **8.3**: Describe how nodes, once connected, choose a blockchain history and receive the historical data
- **8.4**: Describe how transactions/messages are verified and propagated throughout the network
- **8.5**: Describe how blocks are verified and propagated throughout the network

Technology Requirements

Required Technology

This project requires the following tools and technologies:

- GitHub (one repository per team)

Suggested Tools & Platforms

- Languages/Frameworks: Solidity, Truffle, Hardhat, Web3.js or Ethers.js
- Blockchains: Ethereum, Polygon, or Hyperledger Fabric
- Storage: IPFS, Filecoin, or encrypted off-chain DB

Description

You will design and implement a **blockchain-powered supply chain provenance system** that ensures transparency, traceability, and trust among participants in a product's lifecycle. The project will demonstrate how blockchain technology can be applied to **track assets, verify authenticity,** and **prevent fraud or tampering** in industries such as agriculture, pharmaceuticals, luxury goods, or logistics.

Directions

Work in teams to design, implement, and evaluate your blockchain-powered supply chain provenance system. This project has four (4) sets of required deliverables, which are all team submissions:

1. Project Proposal and Team Contract
2. Smart Contract Design Draft
3. Interim Demo
4. Final Project Deliverables

Specific instructions for each deliverable are provided in separate overview documents which are located in the "Project Overviews and Resources" page in the *Module 0: Welcome and Start Here* module. Please review the details in advance so you can work effectively on your coursework.

Project Components

Your team's Blockchain-Based Supply Chain Provenance System Project must include the development of a system design, implementation, and analysis. Each phase should build upon the previous one and demonstrate the progression of your system from concept to evaluation.

All teams must use a shared public **GitHub** repository as the central workspace for collaboration, version control, documentation, and code management. The repository should be actively maintained and organized throughout the course, serving as the official record of your project development. For deliverables that require it, you will submit the repository link to demonstrate your team's progress and ongoing work.

System Design

1. Identify **Stakeholders**
    a. Typical stakeholders include:
        i. Producers / Manufacturers: Create and register products on the blockchain, initiating the provenance record.
        ii. Suppliers / Distributors: Handle logistics, update shipment and custody information, and record transfer events.
        iii. Retailers: Receive goods, verify authenticity, and update product availability or condition.
        iv. Regulators / Certifying Authorities: Oversee compliance, issue approvals or certifications, and audit on-chain records.
        v. Consumers: Access and verify product origin and authenticity through a user interface (e.g., QR code or dashboard).
    b. For each stakeholder:
        i. Define their data access level (read/write).
        ii. Specify events or transactions they generate or validate.
        iii. Indicate how they interact with the smart contract (e.g., registration,
        iv. transfer, verification).
2. Define the **Product Journey**
    a. A typical product journey may include:

i. **Creation**: The producer or manufacturer registers the product on the blockchain with a unique identifier and relevant production details (e.g., batch ID, date, origin).

ii. **Shipment**: The product is dispatched to a distributor or retailer. A transaction is recorded to capture the custody transfer, shipment date, and transport details.

iii. **Storage**: The product is received and stored by an intermediary or retailer. Storage conditions, verification checks, or location updates may be logged on-chain.

iv. **Delivery**: The product reaches the end consumer or final destination. The delivery confirmation and final ownership transfer are recorded on the blockchain.

b. Link each event to the responsible stakeholder.

3. Model **Transactions** and **Events**

   a. Typical examples include:

      i. Product Registration: Records the creation of a new product with a unique ID and initial metadata.

      ii. Ownership / Custody Transfer: Captures changes in responsibility as the product moves between stakeholders.

      iii. Status or Condition Update: Logs important product state changes such as “Originated”, “Shipped”, “Received” or “Delivered.”

      iv. Verification or Validation: Confirms authenticity, quality checks, or regulatory approvals tied to the product ID.

4. Design **Smart Contracts**

   a. Define data structures for product information (ID, owner, status, metadata etc.).

   b. Implement functions for key operations such as registration, transfer, and status updates.

   c. Emit events to record major state changes (e.g., ProductCreated, OwnershipTransferred).

   d. Ensure immutability and verifiability of critical records.

   e. Consider error handling and safeguards against invalid or duplicate transactions.

5. Design **Access Control Mechanism**

   a. Identify roles such as producer, distributor, retailer, regulator, and consumer.

b. Implement role-based access control (RBAC) within smart contracts using modifiers or access control libraries.

c. Prevent unauthorized actions such as tampering with records or reassigning ownership.

Implementation

1. Use **Solidity + Ethereum** (or Hyperledger Fabric) to develop the blockchain layer.
2. Smart contracts should handle at least:
    a. Product registration and unique ID assignment.
    b. Status updates and event logging.
    c. Ownership or custody transfer between entities.
3. Implement a **simple user interface** (web or command-line) for submitting and viewing product data.
4. Optionally **integrate off-chain data** (e.g. IPFS for certificates, simulated IoT data).

Analysis

1. Evaluate scalability, gas cost, and data management tradeoffs.
2. Discuss real-world implementation challenges (privacy, interoperability, regulation).
3. Compare blockchain-based tracking to conventional centralized supply chain systems.

Example Application Ideas

- **Tokenized Asset Tracking System**: Represent each product batch as a unique on-chain token that updates ownership and state across the supply chain.
- **IoT-Enabled Logistics Chain**: Integrate blockchain with simulated IoT data to record real-time shipment conditions via oracles.
- **Multi-Tier Provenance Tracking**: Trace sub-suppliers across multiple tiers to reveal full product lineage and ESG compliance.
- **Hybrid On-Chain/Off-Chain Provenance**: Store detailed records in IPFS or databases while recording cryptographic hashes on-chain.
- **Autonomous Shipment Verification**: Use smart contracts to auto-confirm deliveries or trigger payments based on oracle data.

Note: Your system should clearly demonstrate how blockchain ensures transparency and accountability in a multi-party supply chain. Focus on practical usability and clear visualization of how trust is achieved through decentralized data recording.